# PROGRAMERING B

Eksamensprojekt - Webapplikation

**Elev:** Oliver van Heezewijk
**Klasse:** 3.R

**Vejleder:** Mikkel Kirkeby Mosthaf

**Afleveringsfrist:** 27/03/2026
**Anslag:** 16.425

## Resume

Dette projekt har til formål at udvikle en webbaseret applikation, der kan hjælpe virksomheder med at analysere og skabe overblik over deres indtjening på tværs af produktkategorier.

Applikationen er udviklet i Python med Flask og anvender en SQLite-database til lagring af data. Systemet gør det muligt at tilføje og se produkter samt sammenligne kategorier ud fra indtjening, antal solgte produkter og gennemsnitspris pr produkt. Derudover visualiseres dataene gennem grafer, hvilket giver en bedre forståelse og et lettere overblik.

Resultatet er en funktionel og brugervenlig webapplikation, som demonstrerer anvendelse af databaser, webudvikling og databehandling i praksis

# Indholdsfortegnelse

**Resume** ..... **2**

**Problemstilling & projektbeskrivelse** ..... **4**

*Produkt* ..... *4*

*Problem* ..... *4*

*Krav til løsning* ..... *5*

*Gruppemedlemmers fokusområder* ..... *5*

**Funktionsbeskrivelse** ..... **6**

*Skitser* ..... *6*

*Funktioner & brugergrænseflade* ..... *8*

**Dokumentation** ..... **9**

*Opbygning & struktur* ..... *9*

*Kodegennemgang* ..... *10*

*Redegørelse for brug af AI* ..... *14*

**Test** ..... **15**

*Pytest* ..... *15*

**Konklusion** ..... **16**

**Bibliografi** ..... **17**

**Bilag** ..... **17**

# Problemstilling & projektbeskrivelse

I de følgende afsnit beskrives produktet, det problem der skal løses, vores tilgang til løsningen samt gruppemedlemmernes fokusområder.

## Produkt

I vores projekt har vi haft til formål at lave en hjemmeside som kan holde styr på forretningers salg af varer/produkter.

Vores sidde har tre overordnet værktøjer: *Tilføj produkter*, *sammenligning mellem to kategorier* og *alle produkter*.

Under tilføj produkter kan man tilføje nye produkter som forretningen af solgt ved at angive følgende data:

- Produktets navn
- Hvilken kategori produktet tilhører (for eksempel møbler)
- Produktets pris
- Antal solgte

Vi har også gjort således at man kan uploade en Excel fil med produkter, som så tilføjes til databasen. Dette er væsentligt hvis man har en stor mængde produkter som man ikke vil tilføje manuelt.

Under sammenligning mellem kategorier kan man se en sammenligning af forskellige produktkategorier og se forskel i salg, antal solgte produkter og gennemsnitsprisen af hvert solgte produkt, hvilket vises med grafer. Dette skal kunne hjælpe forretninger med at se hvor de klare sig bedst og hvor der er plads til optimering.

Under alle produkter vises en oversigt over alle forretnings solgte produkter. Her er de samlet i skemaer med andre produkter som tilhører samme kategori. I toppen af siden er der en søgefunktion som gør det muligt at søge efter specifikke produkter eller kategorier.

## Problem

Mange virksomheder mangler et klart overblik over deres indtjening og produktperformance. Data er ofte uoverskuelig eller spredt, hvilket gør det svært at identificere styrker og svagheder. Det fører til dårligere beslutninger og uudnyttet optimeringspotentiale.

## Krav til løsning

For at løse det opstillede problem er det nødvendigt, at vores hjemmeside opfylder en række krav, så løsningen fungerer korrekt og effektivt. Kravene er opstillet i skemaet nedenfor:

| |
|---|
| **Brugeren skal kunne tilføje solgte produkter til databasen.** |
| **Brugeren skal kunne slette produkter fra databasen.** |
| **Alle solgte produkter skal vises struktureret i tabeller opdelt efter kategori.** |
| **Brugeren skal kunne søge efter specifikke produkter eller en bestemt kategori.** |
| **Systemet skal kunne udregne samlet indtjening af alle solgte produkter.** |
| **Brugeren skal kunne sammenligne to kategorier med henblik på: omsætning, antal solgte produkter, gennemsnitspris pr. produkt.** |
| **Fokus på usability og overskuelighed af hjemmesiden.** |
| **Koden skal være opdelt efter trelagsmodellen for at sikre god struktur** |

## Gruppemedlemmers fokusområder

I vores projekt har vores fokusområder hver især set således ud:

Oliver
- Database-design (relationel database)
- Pivot-lignende sammenligning af kategorier
- Frontend-udvikling

Nuftalem
- CRUD-operationer (Create, Read, Update, Delete)
- Databehandling
- Strukturering af kode (funktioner/klasser)

Vi har arbejdet med Liveshare og Github.

# Funktionsbeskrivelse

I de følgende afsnit beskrives skitseprocessen med begrundelser og overvejelser gennem processen. Hvorefter det endelige produkt vises med funktionsbeskrivelse.

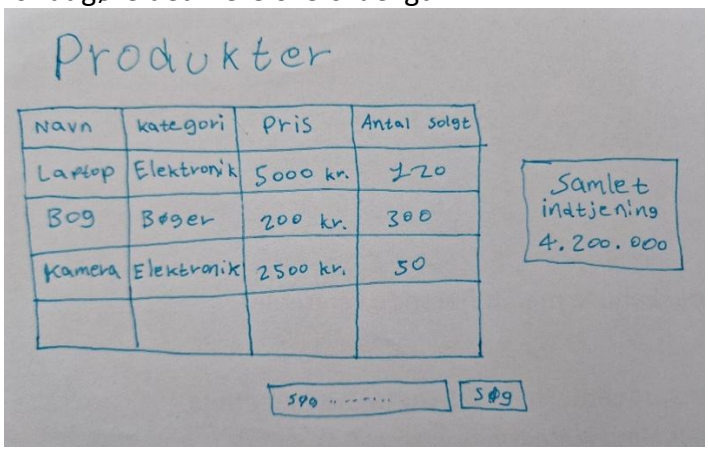
## Skitser

**Alle produkter**

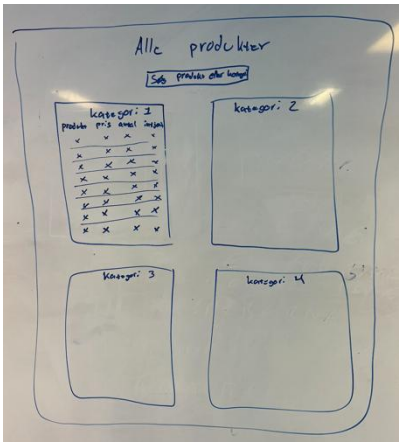
I vores første udkast ville vi vise alle de solgte produkter i en tabel, samt den samlet indtjening til højre for tabellen også skulle søgefunktionen være i bunden.

Efter at have testet det fandt vi ud af at dette layout ikke var optimalt overhoved. Tabellen virkede rodet når der var mange produkter. Søgefunktionens placering i bunden var især et dumt valg for man var nødsaget til at scroll gennem hele tabellen for at nå ned til den.

Det andet udkast er det som vi har valgt at gå med. Vi har søgefunktionen i toppen for at fremme usability. Samtidig har vi lavet tabeller til hver kategori i stedet for en enkelt tabel til alle produkter, for at gøre det mere overskueligt.



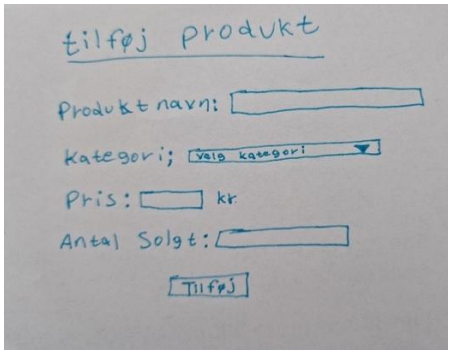

*Figur 2. Første udkast af Alle produkter siden*




*Figur 1. Andet udkast af Alle produkter siden*

**Tilføj produkter**

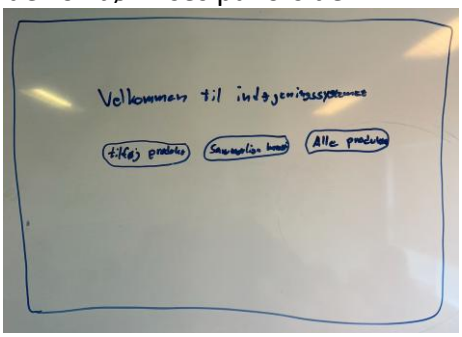
Vi har valgt at gå med den første skitse vi lavet som ses nedenfor. Det er sat op således at produktnavnet står i toppen, så er der en dropdown menu hvor man kan vælge hvilken kategori produktet tilhører. Som det næste skal man angive prisen på produktet og til sidst hvor mange af dem som man har solgt.



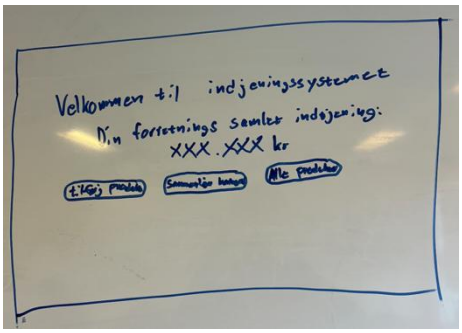

*Figur 3. Første udkast af Tilføj produkter*

**Forside**

Vores første udkast var meget simpel, den indeholder 3 knapper som dirigerer brugeren videre til de forskellige værktøjer samt en overskrift. I vores andet udkast har vi tilføjet at forretningens samlet indtjening vises på forsiden, det synes vi var relevant af flere årsager, forsiden havde ikke meget indhold til at starte med også mener vi at forretningens indtjening er en vigtig information og derfor bør vises på forsiden.



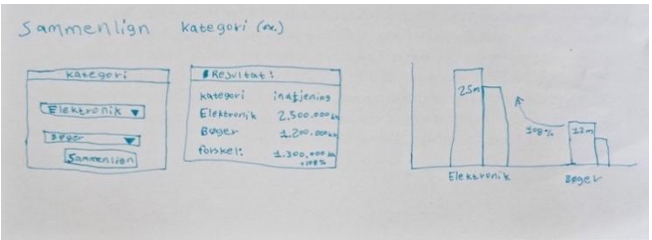

*Figur 5. Første udkast af hjemmesidens forside*




*Figur 4. Andet udkast af hjemmesidens forside*

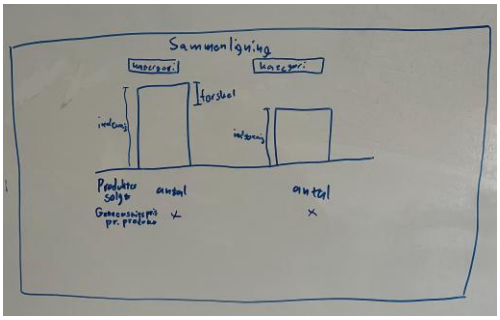
**Sammenligning mellem kategorier**

I vores første udkast havde vi en boks en boks hvor man angav hvilke to kategorier som man vil sammenligne, man valgte mellem kategorierne via en dropdown menu. Ved siden af havde vi en boks som viste de forskellige kategoriers indtjening samt forskellen mellem dem. Til sidste var graferne som skulle vise dataen visuelt.



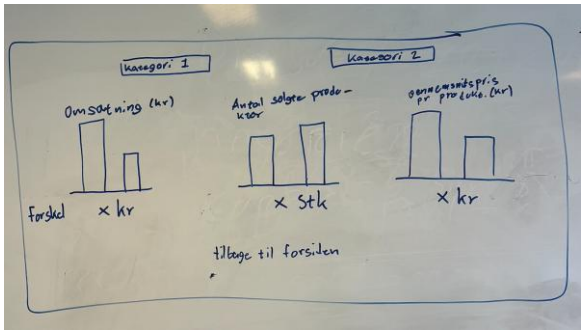

*Figur 6. Første udkast af Sammenligning mellem kategorier*

Dette layout var langt fra optimalt, alt kunne ikke være ved siden af hinanden og vi ville gerne vise mere relevant data. I andet udkast havde vi valgt at sætte kategori dropdown menuerne i toppen af siden i stedet for i en boks, hvilket ikke tog lige så meget plads og samtidig gjorde dem let tilgængelige. Så havde vi flyttet diagrammet sådan at det var placeret centralt på siden. Nedenfor havde vi tilføjet nyt data som viste hvor mange produkter var solgt af hver kategori samt gennemsnitsprisen for produkter solgt.




*Figur 7. Andet udkast af Sammenligning mellem kategorier*

I vores tredje og sidste udkast havde vi valgt at tilføje således at den nye data fra udkast to også blev vist visuelt i diagrammer også vist forskellen mellem kategorierne nedenfor. Vi har valgt at gå med dette layout grundet at alt dataen er visualiseret og siden er meget simpel og overskuelig.

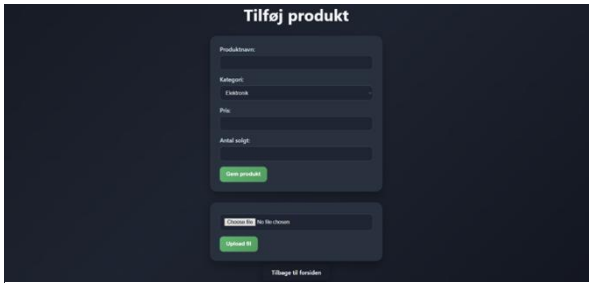



*Figur 8. Tredje udkast af Sammenligning mellem kategorier*

## Funktioner & brugergrænseflade

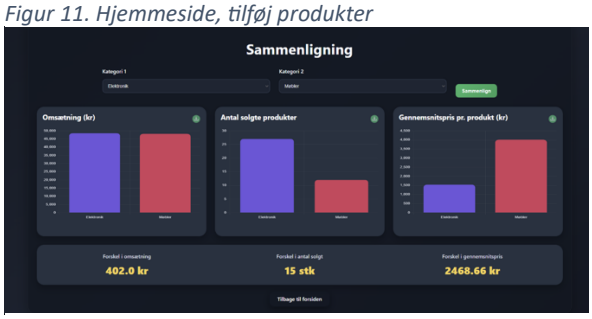
*Figur 12. Hjemmeside, forside*

På forsiden bliver man mødt af en velkomstbesked og en oversigt over ens forretnings samlet indtjening.
Nedenfor er der tre knapper som diagerer brugeren til de andre sider af hjemmesiden.




*Figur 11. Hjemmeside, tilføj produkter*

På denne side kan man tilføje produkter til oversigten. Det er muligt at gøre det et produkt ad gangen eller ved at uploade et Excel dokument.
I et scenarie hvor man havde en rigtig forretning ville vi naturligvis lave det sådan at produkterne automatisk blev tilføjet til oversigten.




*Figur 10. Hjemmeside, sammenligning mellem produkter*

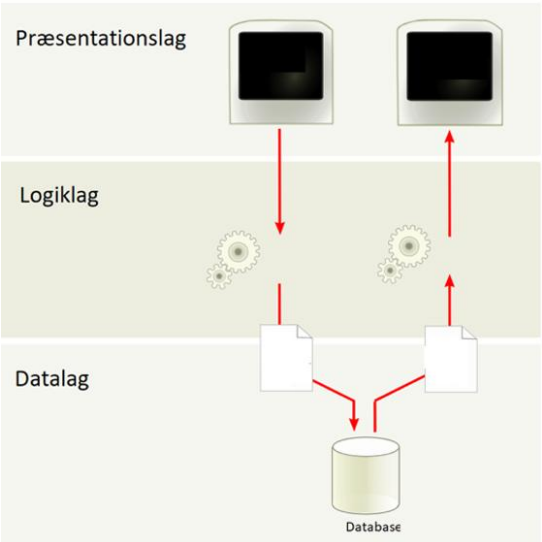
På denne side kan man sammenligne salget mellem to kategorier. For begge kategorier vises omsætning, antal solgte produkter og gennemsnitspris pr. produkt som grafer sådan at man visuelt kan se forskellen. Nedenfor graferne kan man læse den præcise forskel.

*Figur 9. Hjemmeside, oversigt over alle produkter*

På denne side vises alle forretningens solgte produkter opdelt i kategorier. Hver gang et nyt produkt tilføjes til hjemmesiden, vises det her.

# Dokumentation

I dette afsnit vil jeg gennemgå vigtigheden af en ordentlig kodestruktur samt beskrive den struktur, vi har anvendt i vores program. Derefter følger en gennemgang af udvalgte kodestykker fra programmet, og til sidst redegøres der for vores brug af kunstig intelligens i forløbet.




*Figur 13. Trelagsmodellen. (Jesper, Bertram, & Damhus, 2017, s. c4731)*

## Opbygning & struktur

Når man har at gøre med store programmer, er det vigtigt at ens struktur er i orden for at kunne holde styr på hele programmet.

Trelagsmodellen er en videreudvikling af klient-server-arkitekturen og beskriver en struktur som er god til at organiserer et program.

Præsentationslaget er det som brugeren ser og interagerer med.

I logiklaget behandles data som brugeren efterspørger eller indsender, fx. via POST- eller GET requests.

I datalaget lagres alle data.

```
PROGRAMMERING-EKSAMEN/
│
├── __pycache__/
├── .pytest_cache/
│
├── static/
│   └── style.css
│
├── præsentation/
│   ├── admin.html
│   ├── allproducts.html
│   ├── index.html
│   └── sammenligning.html
│
├── tests/
│   ├── __pycache__/
│   ├── test_app.py
│   └── test_entity.py
│
├── logik/
│   ├── app.py
│   └── entity.py
│
└── data/
    ├── database.py
    └── products.db
```

*Figur 14. Strukturdiagram over vores kode.*

I vores projekt har vi fulgt trelagsmodellen for at organiserer og strukturerer vores kode ordentligt. Til venstre vises et strukturdiagram over vores kode, hvor præsentations-, logik- og datalagene alle er vist.

## Kodegennemgang

Nedenfor har jeg opstillet en detaljeret gennemgang af den kode som ligger bag sammenligning siden.

```
53  @app.route("/sammenligning", methods=["GET", "POST"])
54  def sammenligning():
55      result = None
```

*Figur 15. Kodeudsnit fra app.py*

Linje 53: En route oprettes i Flask. Sådan at når brugeren går ind på /sammenligning siden køres funktionen sammenligning():

Linje 54: Funktionen sammenligning(): defineres, hvilket er den funktion som styrer logikken bag siden.

Linje 55: Result sættes til none. Det er vigtigt at variablen starter ud med at være tom.

```
57      if request.method == "POST":
58
59          category1 = request.form["category1"]
60          category2 = request.form["category2"]
```

*Figur 16. Kodeudsnit fra app.py*

Linje 57: Der tjekkes om brugeren har indtastet data.

Linje 59: Værdien som brugeren indtastet i første felt gemmes som variabel category1

Linje 60: Værdien som brugeren indtastet i andet felt gemmes som variabel category2

```
62          rows1 = get_products_by_category(category1)
63          rows2 = get_products_by_category(category2)
```

*Figur 17. Kodeudsnit fra app.py*

Linje 62: get_products_by_category kaldes med category1 som argument. For at hente alle produkter fra category1.

Linje 63: get_products_by_category kaldes med category2 som argument. For at hente alle produkter fra category2.

```
65          products1 = [Product(*p[1:]) for p in rows1]
66          products2 = [Product(*p[1:]) for p in rows2]
```

*Figur 18. Kodeudsnit fra app.py*

Linje 65: Listen products1 oprettes hvor alle objekter fra klassen Product i brugerens valgte kategori gemmes, undtagen product name, det sorteres fra da det ikke skal bruges.

Linje 66: Det samme gøres for den anden valgte kategori

```
68          sold1 = sum(p.quantity for p in products1)
69          sold2 = sum(p.quantity for p in products2)
```

*Figur 19. Kodeudsnit fra app.py*

Linje 68: Det samlet antal solgte produkter beregnes for den første valgte kategori.

Linje 69: Det samlet antal solgte produkter beregnes for den anden valgte kategori.

```
71          revenue1 = round(sum(p.revenue() for p in products1), 2)
72          revenue2 = round(sum(p.revenue() for p in products2), 2)
```

*Figur 20. Kodeudsnit fra app.py*

Linje 71: Den samlet omsætning for den første valgte kategori beregnes.

Linje 72: Den samlet omsætning for den anden valgte kategori beregnes.

```
74          avg_price1 = round(
75              sum(p.price for p in products1) / len(products1), 2
76          ) if products1 else 0
78          avg_price2 = round(
79              sum(p.price for p in products2) / len(products2), 2
80          ) if products2 else 0
```

*Figur 21. Kodeudsnit fra app.py*

Linje 74-76: Her beregnes gennemsnitsprisen pr. produkt for produkterne i products1

Linje 78-80: Her beregnes gennemsnitsprisen pr. produkt for produkterne i products1

```
82          result = {
83              "category1": category1,
84              "sold1": sold1,
85              "revenue1": revenue1,
86              "avg_price1": avg_price1,
87              "category2": category2,
88              "sold2": sold2,
89              "revenue2": revenue2,
90              "avg_price2": avg_price2,
91              "revenue_difference": abs(revenue1 - revenue2),
92              "sold_difference": abs(sold1 - sold2),
93              "avg_price_difference": abs(avg_price1 - avg_price2)
94          }
```

*Figur 22. Kodeudsnit fra app.py*

Linje 82-94: Der oprettes et dictionary som indeholder alle de værdier som tidligere i koden er blevet oprettet. Dictionaryet gemmes i variablem result, dette gør man for at de nemt kan sendes videre til HTML-skabelonen.

```
96          return render_template("sammenligning.html", result=result)
```

*Figur 23. Kodeudsnit fra app.py*

Linje 96: HTML siden returneres med alt den nye data og siden vises for brugeren.

```
53  @app.route("/sammenligning", methods=["GET", "POST"])
54  def sammenligning():
55      result = None
56
57      if request.method == "POST":
58
59          category1 = request.form["category1"]
60          category2 = request.form["category2"]
61
62          rows1 = get_products_by_category(category1)
63          rows2 = get_products_by_category(category2)
64
65          products1 = [Product(*p[1:]) for p in rows1]
66          products2 = [Product(*p[1:]) for p in rows2]
67
68          sold1 = sum(p.quantity for p in products1)
69          sold2 = sum(p.quantity for p in products2)
70
71          revenue1 = round(sum(p.revenue() for p in products1), 2)
72          revenue2 = round(sum(p.revenue() for p in products2), 2)
73
74          avg_price1 = round(
75              sum(p.price for p in products1) / len(products1), 2
76          ) if products1 else 0
77
78          avg_price2 = round(
79              sum(p.price for p in products2) / len(products2), 2
80          ) if products2 else 0
81
82          result = {
83              "category1": category1,
84              "sold1": sold1,
85              "revenue1": revenue1,
86              "avg_price1": avg_price1,
87              "category2": category2,
88              "sold2": sold2,
89              "revenue2": revenue2,
90              "avg_price2": avg_price2,
91              "revenue_difference": abs(revenue1 - revenue2),
92              "sold_difference": abs(sold1 - sold2),
93              "avg_price_difference": abs(avg_price1 - avg_price2)
94          }
95
96      return render_template("sammenligning.html", result=result)
```

*Figur 24. Sammenligning funktion, samlet kode.*

Gennemgang af koden bag vores klasse:

```
1  class Product:
2      def __init__(self, name, category, price, quantity):
3          self.name = name
4          self.category = category
5          self.price = price
6          self.quantity = quantity
7
8      def revenue(self):
9          return self.price * self.quantity
```

*Figur 25. Kodeudsnit fra entity.py*

Linje 1: Her defineres klassen med navnet Product.

Linje 2: Klassen constructor som har parametrene name, category, price og quantity.

Linje 3: Her oprettes en instansvariabel name, og den får værdien fra parameteren name.

Linje 4: Her oprettes instansvariablen category, som gemmer produktets kategori

Linje 5: Her oprettes instansvariablen price, som gemmer produktets pris.

Linje 6: Her oprettes instansvariablen quantity, som gemmer hvor mange der er solgt.

Linje 8: Der der defineres en funktion i klassen, til at regne omsætningen af produkter.

Linje 9: Produktets omsætning retuneres.

Gennemgang af koden som opretter databasen:

```
8   def create_database():
9       conn = get_db()
10      cursor = conn.cursor()
```

*Figur 26. Kodeudsnit fra database.py*

Linje 8: Funktionen create_database(): defineres.

Linje 9: funktionen get_db kaldes, som returnerer en databaseforbindelse.

Linje 10: Her oprettes en cursor. Den bruges til at udføre SQL-kommandoer på databasen

```
12      cursor.execute("""
13      CREATE TABLE IF NOT EXISTS products (
14          id INTEGER PRIMARY KEY AUTOINCREMENT,
15          name TEXT NOT NULL,
16          category TEXT NOT NULL,
17          price REAL NOT NULL,
18          quantity INTEGER NOT NULL
19      )
20      """)
```

*Figur 27. Kodeudsnit fra database.py*

Linje 13: Funktionen opretter en tabel hvis der ikke allerede findes en.

Linje 14: Opretter et kolonne id.

Linje 15: Sikre at name er en tekstværdi og ikke null.

Linje 16: Sikre at category er en tekstværdi og ikke null

Linje 17: Sikre at price er et decimaltal.

Linje 18: Sikre at quantity er et heltal

```
22        conn.commit()
23        conn.close()
```

*Figur 28. Kodeudsnit fra database.py*

Linje 22: Ændringerne i databasen gemmes.

Linje 23: Databaseforbindelsen lukkes.

Gennemgang af CSS:
Udseendet af hjemmesiden er lavet ved hjælp af CSS. CSS bruges til at bestemme elementers placering, farver, størrelser og generelle layout. Man kan bruge CSS-selectors, til at bestemme hvordan bestemte elementer i HTML skal se ud. Nedenfor har jeg to eksempler af CSS-selectors som vi bruger på hjemmesidens forside.

```
393  .revenue {
394      font-size: 2.8rem;
395      font-weight: 700;
396      color: #1296b7;
397      margin-top: 10px;
398  }
```

*Figur 30. Kodeudsnit fra style.css*

```
50  .nav {
51      display: flex;
52      justify-content: center;
53      gap: 24px;
54      margin-top: 30px;
55      flex-wrap: wrap;
56  }
```

*Figur 29. Kodeudsnit fra style.css*

Linje 393 & 50: CSS-selectors som bestemmer hvordan elementer skal se ud hvis de er omfattet af selectoren.

Linje 51: display er en CSS property og bestemmer hvordan elementerne skal placere sig på siden.

Linje 52: justify er en CSS property bestemmer hvordan elementerne på siden skal være placeret.

Linje 396: Color er en CSS property bestemmer hvilken farve teksten skal være.

Disse er blot få af mange CSS propertys. Der er flere hundred forskellige som alle gør forskellige ting (MDN contributors, 2025).

## Redegørelse for brug af AI

I dette projekt har vi brugt kunstig intelligens (AI) som et hjælpemiddel i udviklingen af vores webapplikation. AI har bidraget med at løse tekniske problemer og støttet os i at forbedre både design og struktur i vores kode.

| Prompt | Formål | Hvordan vi brugte det |
|---|---|---|
| Vi gav AI en samlet instruktionsfil om at udvikle en webapplikation i Flask til at starte med | At skabe en samlet struktur og retning for hele projektet | Vi brugte AI's forslag som udgangspunkt for projektets opbygning og videreudviklede selv funktionerne trin for trin |
| Lav en Flask app med SQLite database, hvor man kan tilføje, vise og slette produkter | At opbygge backend og database | Vi brugte det som grundstruktur og tilpassede koden til vores egen app |
| Hvordan kan jeg sortere produkter alfabetisk i Flask med SQLite | At gøre data mere overskueligt | Vi implementerede ORDER BY i vores SQL-queries |
| Hvordan kan jeg vise produkter opdelt i kategorier i Flask og HTML templates | At strukturere data i frontend | Vi delte produkter op i Elektronik, Møbler, Personlig pleje og Tøj |
| Lav en funktion i Flask der sammenligner to kategorier og beregner samlet indtjening | At lave en analysefunktion | Vi brugte SUM (price * quantity) til at sammenligne kategorier |
| Lav en HTML side med to kolonner og grafer til at sammenligne data visuelt | At forbedre brugeroplevelsen | Vi designede en sammenligningsside med grafer og visuelle elementer |
| Flyt søgefunktionen fra forsiden til en separat side i Flask appen | At gøre forsiden mere simpel | Vi flyttede søgefeltet til "Alle produkter" siden |
| Giv mig moderne CSS til en webapp med dark theme, kort-design og flotte knapper | At forbedre design og UI | Vi implementerede et dark theme med cards og gradients |
| Hvordan laver jeg en centreret forside med tre knapper og velkomsttekst | At forbedre layout og førstehåndsindtryk | Vi brugte flexbox til at centrere indholdet |
| Hvorfor får jeg fejl i min HTML med Jinja2, og hvordan retter jeg dem | At fejlrette kode | Vi rettede syntax-fejl og template-problemer |
| Forbedr mit CSS så tabeller og layout ser moderne og professionelt ud | At gøre designet mere professionelt | Vi forbedrede spacing, farver og hover-effekter |

AI har været et nyttigt værktøj i vores udviklingsproces, men vi har selv stået for at forstå, tilpasse og implementere løsningerne. Vi har også testet koden løbende, da AI ikke altid giver fejlfrie svar.

# Test

For at sikre om vores program fungere korrekt, har vi ved hjælp af AI opsat en række pytests som gennemgår vores kode og tjekker om alt er funktionelt.

## Pytest

pytest er et testframework til Python, der bruges til at teste, om kode virker korrekt. Frameworket giver detaljeret feedback ved fejl, så man hurtigt kan se, hvad der er galt. pytest kan bruges til både små og store projekter (krekel & team, 2015).

Vores pytest tester, om vores Flask-app og database fungerer korrekt.

Testene tjekker:

- at sider/routes virker
- at produkter kan oprettes og slettes i databasen
- at søgning fungerer korrekt
- at beregninger giver de rigtige resultater

Pytest sikre at både funktionalitet og logik i programmet virker som forventet.

```
PS C:\Users\Nuftalem\OneDrive\Documents\GitHub\Programmering-eksamen> python -m pytest -v
========================================= test session starts =========================================
platform win32 -- Python 3.12.10, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\Nuftalem\AppData\Local\Microsoft\
WindowsApps\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Nuftalem\OneDrive\Documents\GitHub\Programmering-eksamen
collected 8 items

tests/test_app.py::test_index_returns_200 PASSED                                                [ 12%]
tests/test_app.py::test_admin_get_returns_200 PASSED                                            [ 25%]
tests/test_app.py::test_admin_post_creates_product_and_redirects PASSED                         [ 37%]
tests/test_app.py::test_delete_product_removes_product_and_redirects PASSED                     [ 50%]
tests/test_app.py::test_search_finds_matching_product PASSED                                    [ 62%]
tests/test_app.py::test_allproducts_returns_200 PASSED                                          [ 75%]
tests/test_app.py::test_sammenligning_calculates_correct_result PASSED                          [ 87%]
tests/test_entity.py::test_product_revenue PASSED                                               [100%]

========================================= 8 passed in 0.45s =========================================
```

## Konklusion

I dette projekt har vi med succes udviklet en webapplikation, der løser problemet med manglende overblik over virksomheders salg og produktdata. Systemet gør det muligt at samle, strukturere og analysere data på en overskuelig måde, hvilket gør det lettere for virksomheder at identificere deres styrker og svagheder.

Vores webapplikation løser alle de opstillet krav og har samtidig særlig visualiseringen af data gennem grafer som bidrager til en bedre forståelse af forskelle og tendenser.

Ved at anvende trelagsmodellen har vi opnået en velstruktureret og overskuelig kode, som gør programmet lettere at vedligeholde og videreudvikle. Test med pytest har yderligere sikret, at både funktionalitet og beregninger fungerer korrekt.

Brugen af AI har været en hjælp i udviklingsprocessen, men det har været nødvendigt selv at forstå og tilpasse løsningerne, hvilket har styrket vores faglige kompetencer.

Samlet set har forløbet gået udmærket. Vores webapplikation kan bruges til at skabe praktiske løsninger, der forbedrer overblik, effektivitet og beslutningsgrundlag hos virksomheder.

## Bibliografi

Jesper, B., Bertram, M. B., & Damhus, M. (2017). *Informatik*. Hentet 20. Marts 2026 fra Informatik.systime.dk: https://informatik.systime.dk/?id=1124

krekel, h., & team, p.-d. (2015). *pytest: helps you write better programs*. Hentet 26. marts 2026 fra docs.pytest.org: https://docs.pytest.org/en/stable/

MDN contributors. (13. November 2025). *CSS reference*. Hentet 26. Marts 2026 fra developer.mozilla.org: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference#:~:text=*%20Web.%20*%20CSS.%20*%20Reference.

## Bilag

Github repository: https://github.com/Ollie0618/Programmering-eksamen.git